

UNIVERSITAT AUTÒNOMA DE BARCELONA

REPORT

Practicum 2: Labelling

Author

Valentin Micu-Hontan 1718971

Darian Balagiu 1719581

Horia Andrei Moraru 1710314

May 15, 2024

1. Introduction

This practice aims to develop an algorithm for automatically labeling images in a clothing catalog by type and color. It focuses on two machine learning techniques: K-means and K-nearest neighbors (KNN). Our main goal can be seen in [Figure 1](#).

1.1. K-Means Algorithm: An unsupervised learning technique that groups data points into a predefined number of clusters (K). Here, clusters represent colours in an image.

- **Problem:** Given an image, automatically determine its dominant colours.
- The algorithm iteratively assigns data points (pixels) to the closest cluster (centroid) and recalculates the centroid based on the assigned points.
- This process continues until the centroids stabilise (convergence).

1.2 K-NN Algorithm: A supervised learning technique well suited for shape type classification. The training set consists of images with pre-assigned garment type labels (e.g., dress, shirt).

- **Problem:** Automatically assign labels (e.g., dress, shirt) to clothing images based on their shape.

During classification, when presented with a new image, K-NN performs the following steps:

- *Feature Extraction:* K-NN operates on numerical features. Here, grayscale pixel values from the image act as features.
- *K Nearest Neighbors:* Given a new image, K-NN finds the K most similar images (nearest neighbours) from the labeled training set based on feature (grayscale pixel) distance.
- *Label Prediction:* The most frequent shape type label among the K nearest neighbours is assigned to the new image.



Figure 1

1.3.Objectives and potential conclusions

The objective of this practice is to implement improvements to the baseline algorithm and compare the new and old results between each other. We will be using various analysis functions (quantitative and qualitative) in order to explain our hypothesis and results obtained. Finally, based on the improvements made and on an in-depth analysis of the results, we will conclude with the limitations of the model and with suggestions to potential future accuracy improvements.

2. Methods of analysis

2.1.Qualitative analysis functions: Functions designed for visually assessing our classifiers.

a. *Retrieval_by_color*: This function evaluates the effectiveness of K-Means color tagging for image retrieval. It accepts a list of color labels and returns a corresponding set of images. Each image returned contains at least one of the specified color labels.

Example: If the input list contains the label "Red," the function would return a set of images identified as containing significant amounts of red.



b. *Retrieval_by_shape*: This function evaluates the effectiveness of KNN shape tagging for image retrieval. It accepts a list of shapes labels and returns a corresponding set of images. So, if you asked for "pants" and "dress" you'd get two lists—one with all the photos with pants and another with all the photos with dress.

Example: If the input list contains the label "pants", the function would return a set of images identified as containing the shape "pants".



c. *Retrieval combined*: This function evaluates the effectiveness of both KNN shape tagging and K-means color tagging for image retrieval. It accepts two lists, one of shapes and one of colors and returns the corresponding set of images.

Example: If the input list contains the label "pants" and "blue" the function would return a set of images identified as pants that are blue.



Note: The returned images are limited to the first 5 elements retrieved by the corresponding functions.

- d. `Create_bar_chart`: Receives the data corresponding to X and Y axes and renders a bar chart
- e. `Create_plot`: Receives the data corresponding to X and Y axes and renders a plot

2.2 *Quantitative analysis functions*: Functions for numerical evaluation of the classifiers

The implemented functions are:

- a. `Kmean_statistics`
- b. `Get_class_accuracy`
- c. `Get_color_accracy`
- d. `Kmeans_accuracy_vs_centroids`
- e. `Knn_samples_vs_accuracy`
- f. `Knn_neighbours_vs_accuracy`

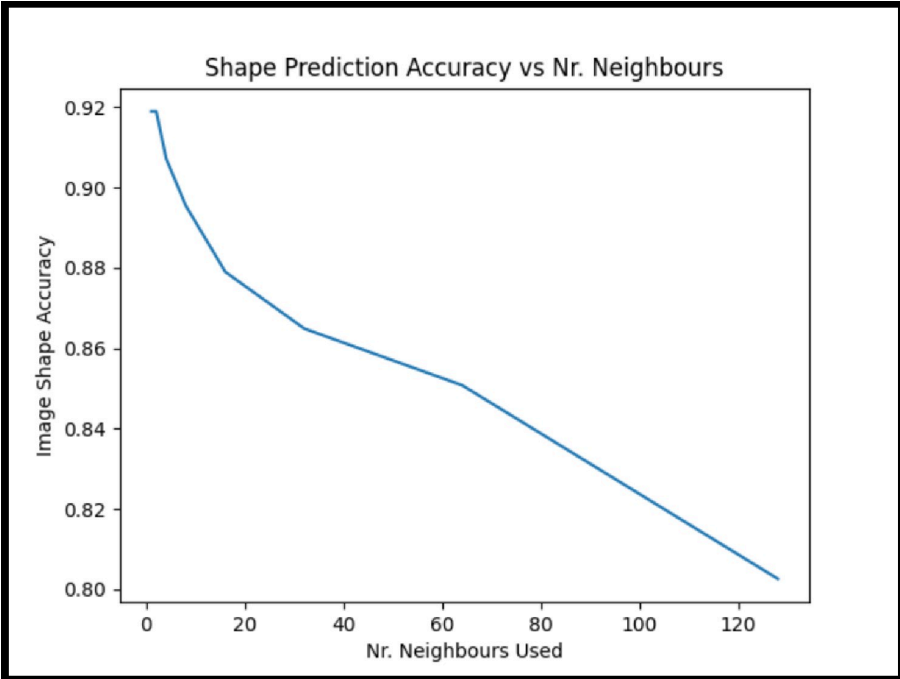
Every function will be mentioned in detail with their corresponding analyses along the review.

3. Baseline results

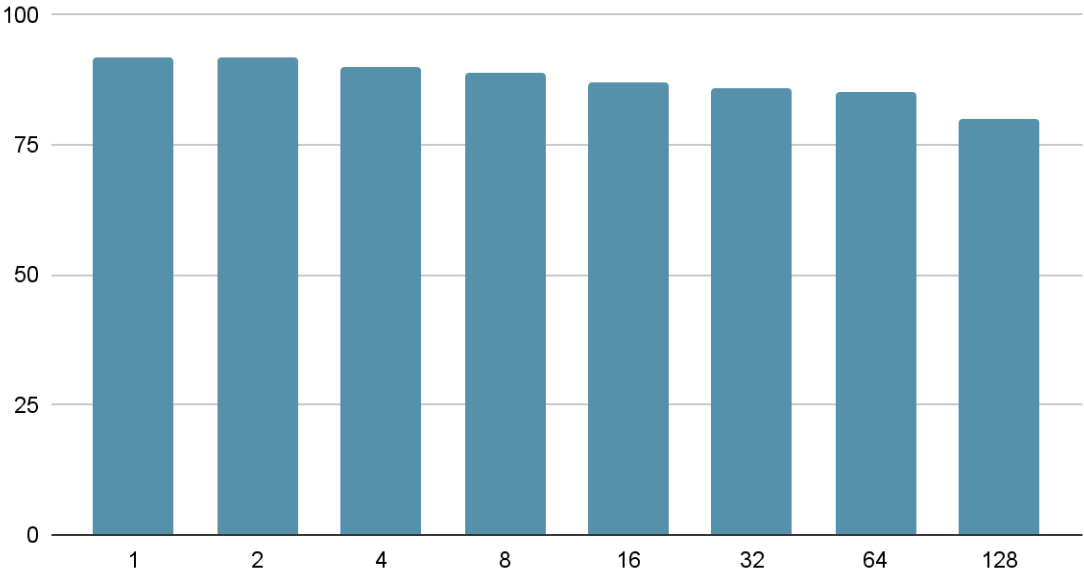
3.1. *Shape Classification*

For the image's shape predictions we used the KNN algorithm, which was trained on 2328 training samples and tested on 851 test samples.

The main parameter of this algorithm is k , which represents the number of closest neighbours that will be taken into consideration when determining the class of the tested sample. Here is a plot that portrays KNN accuracy against the number of neighbours used:

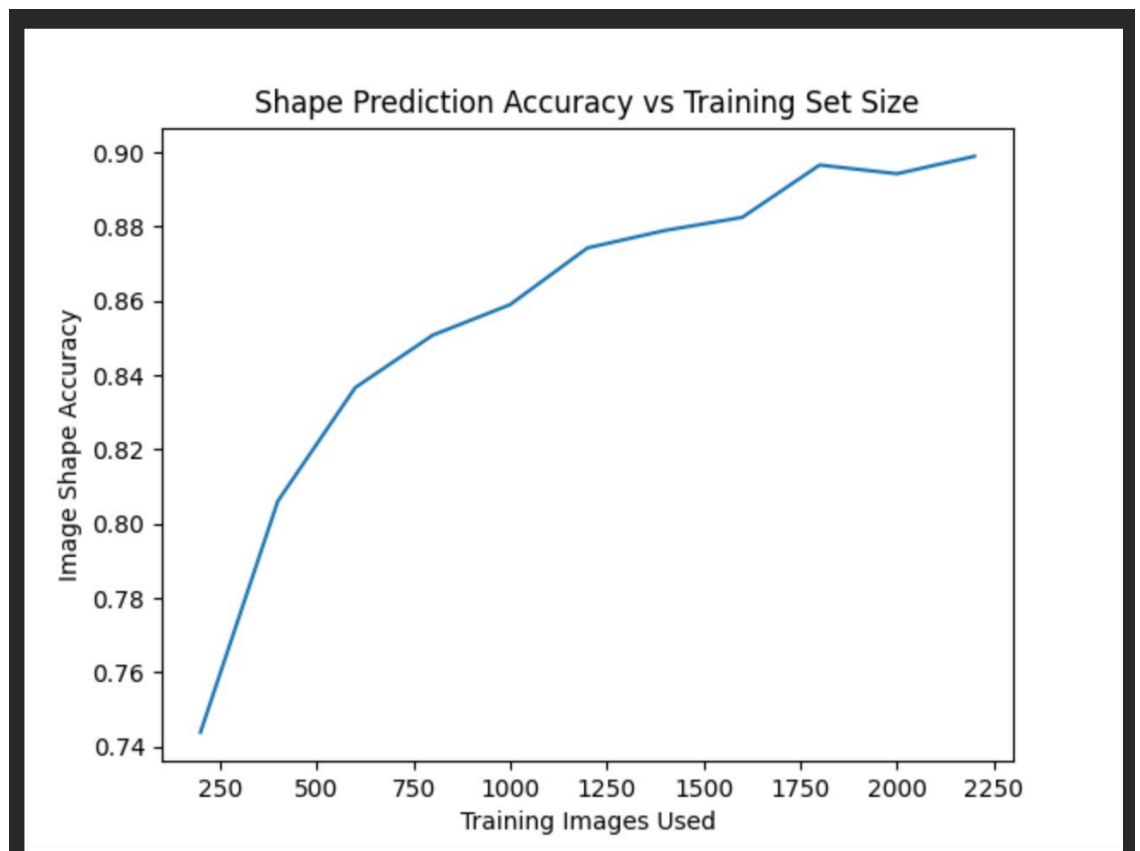


KNN Accuracy vs Neighbours



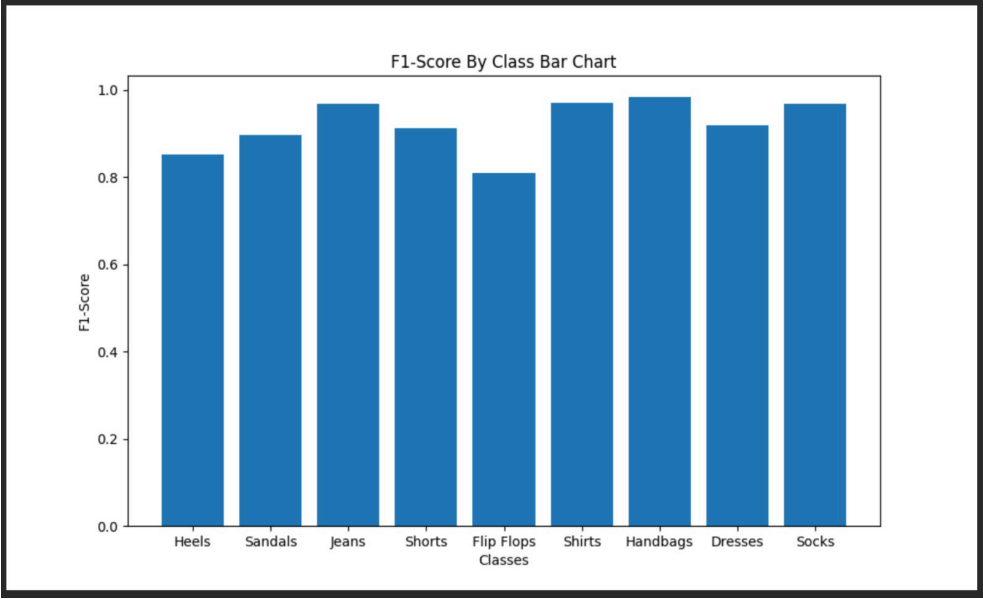
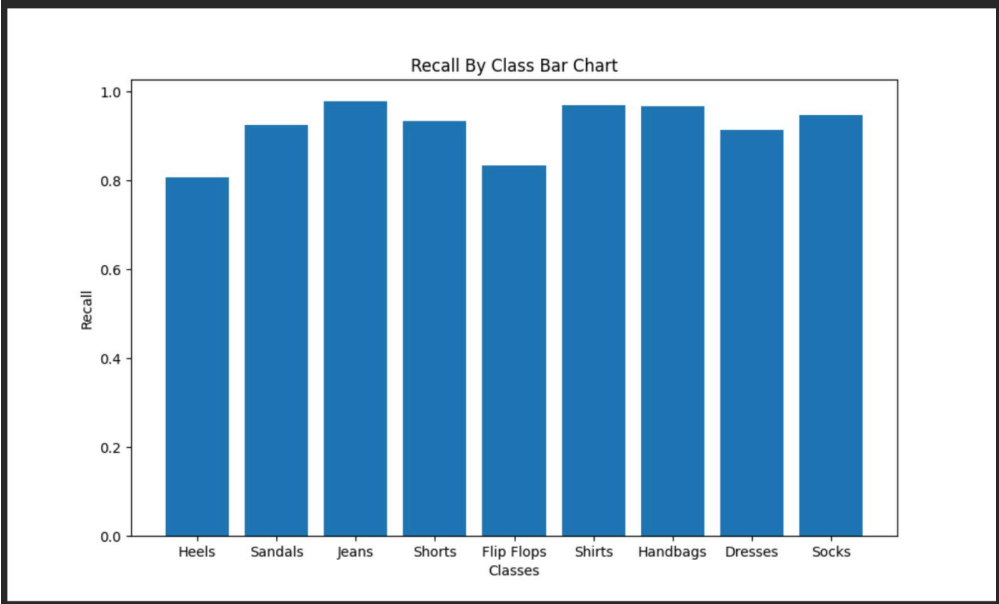
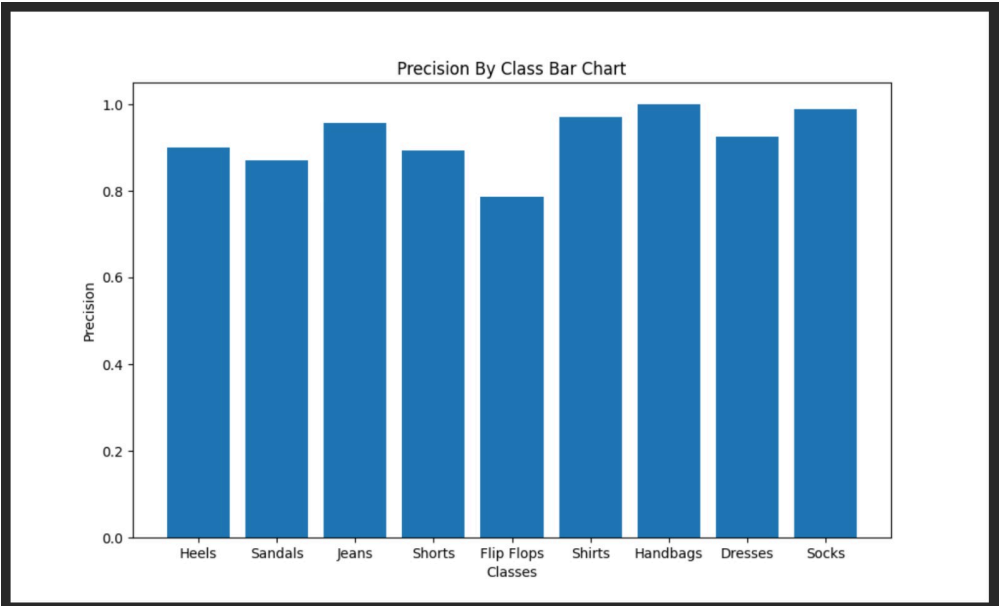
The best accuracy was obtained for $k = \{1, 2\}$: 92%

As can be seen above, the accuracy of the classifier decreases as the number of neighbours used increases. This may be due to the fact that some images learnt from the training set are repeated in the test set or due to the high similarities between some pairs of images. However, another important factor is the number of training samples that the model has learned.

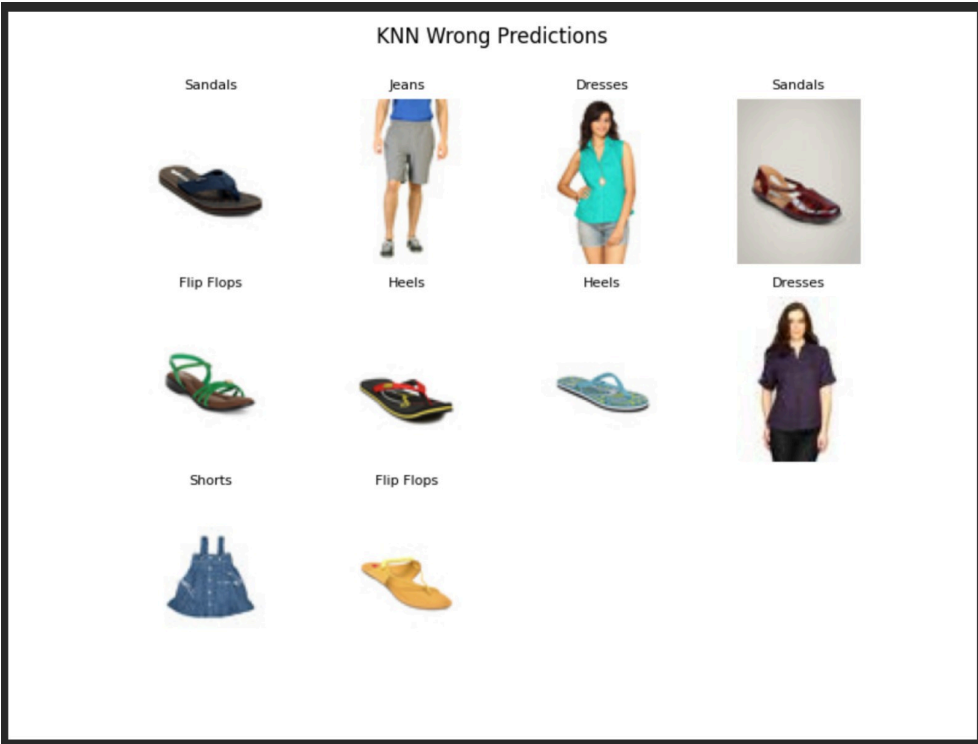
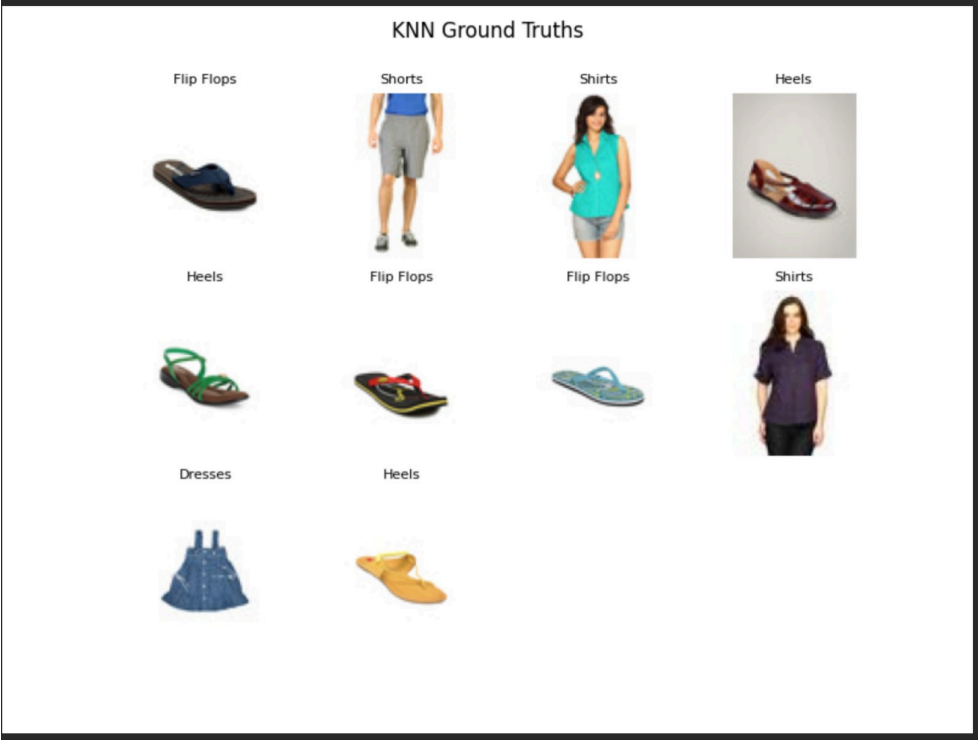


As can be seen above, there is an obvious trend that expresses that the accuracy of the model grows proportionally with the numbers of training samples used. That is because the model sees more samples of the same category multiple times and is able to reduce the noise produced from similar categories (such as Flip-flops vs Sandals). Although the higher the training set the better the performance of the classifier, this also results in a higher complexity of the program and therefore a longer time needed for running.

In order to get a better understanding of the model, we are going to dive into amore particular approach and separately analyse each class' performance. Below we have the Precision, Recall and F1-Score for each of the 8 classes:



As can be seen from the metrics displayed above, the best performing classes are the ones that are not similar to any other classes (such as “Handbags” and “Socks”), while the worst performing are the ones that are similar with each other (such as “Sandals”, “Heels” and ”Flip-Flops”). When computing some of the miss-classified items in the test set, this is the obtained result:

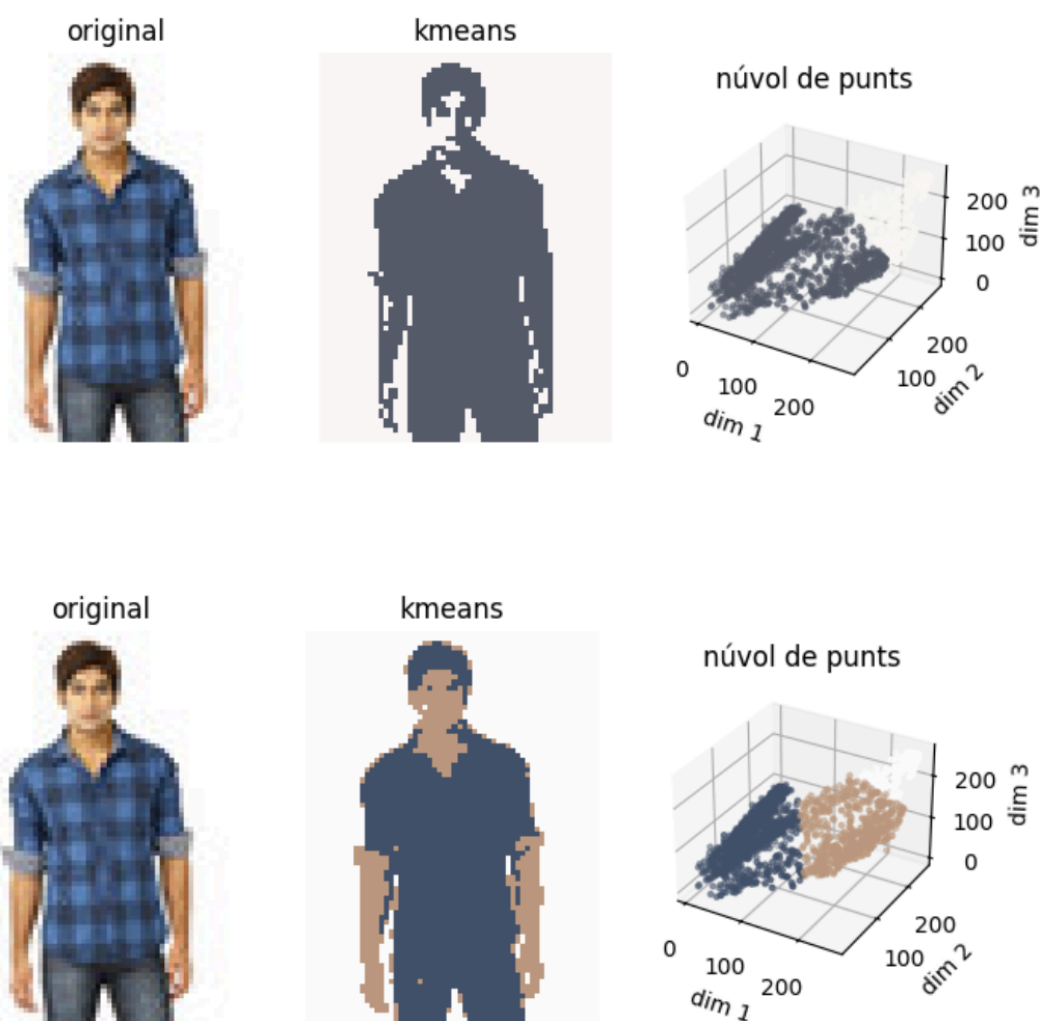


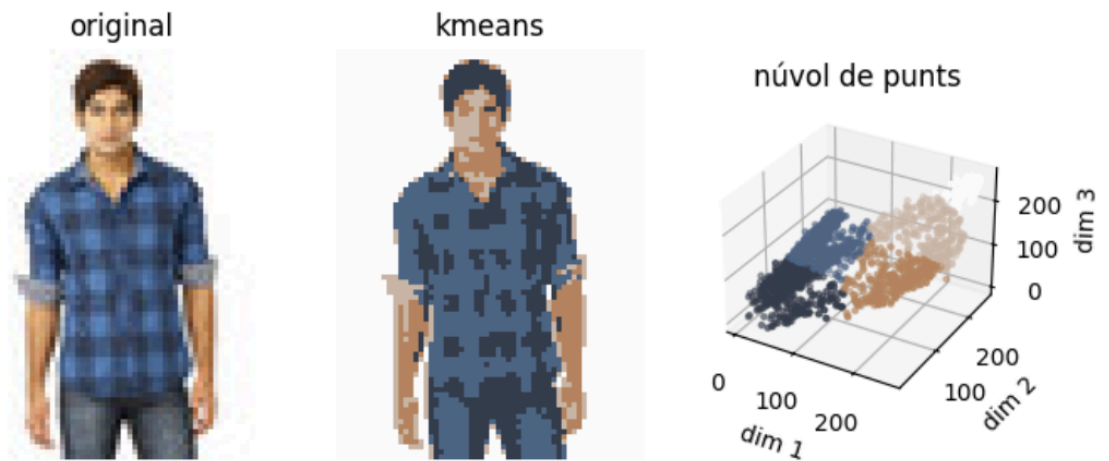
Not to our surprise, 6 out of 10 usual mistakes are coming from miss-classifications of Sandals, Heels and Flip-Flops, due to their similarity.

3.2. Predicting the colours

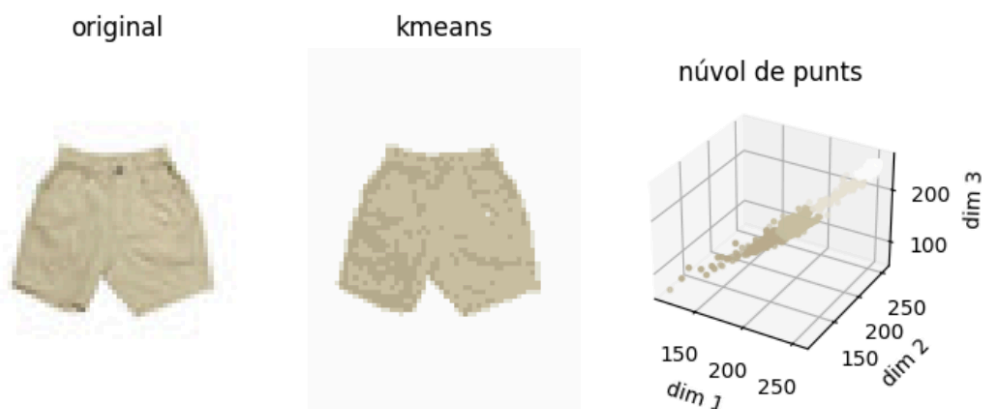
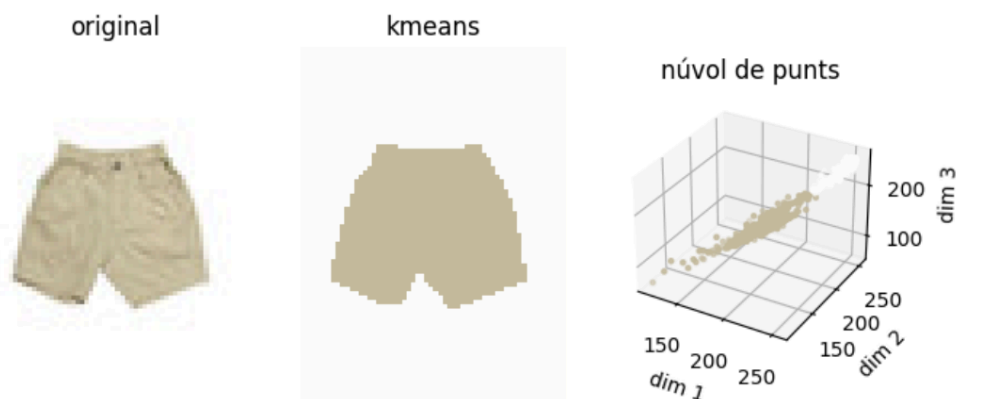
The algorithm we use for predicting the colours of a clothing article is an unsupervised one, KMeans. It transfers the RGB value of each pixel into a 3 dimensional space and then forms clouds of points around some centroids.

Each centroid corresponds to one of the 11 base colours and the number of centroids that we use determine the complexity of the replicated figure. An example can be seen below:

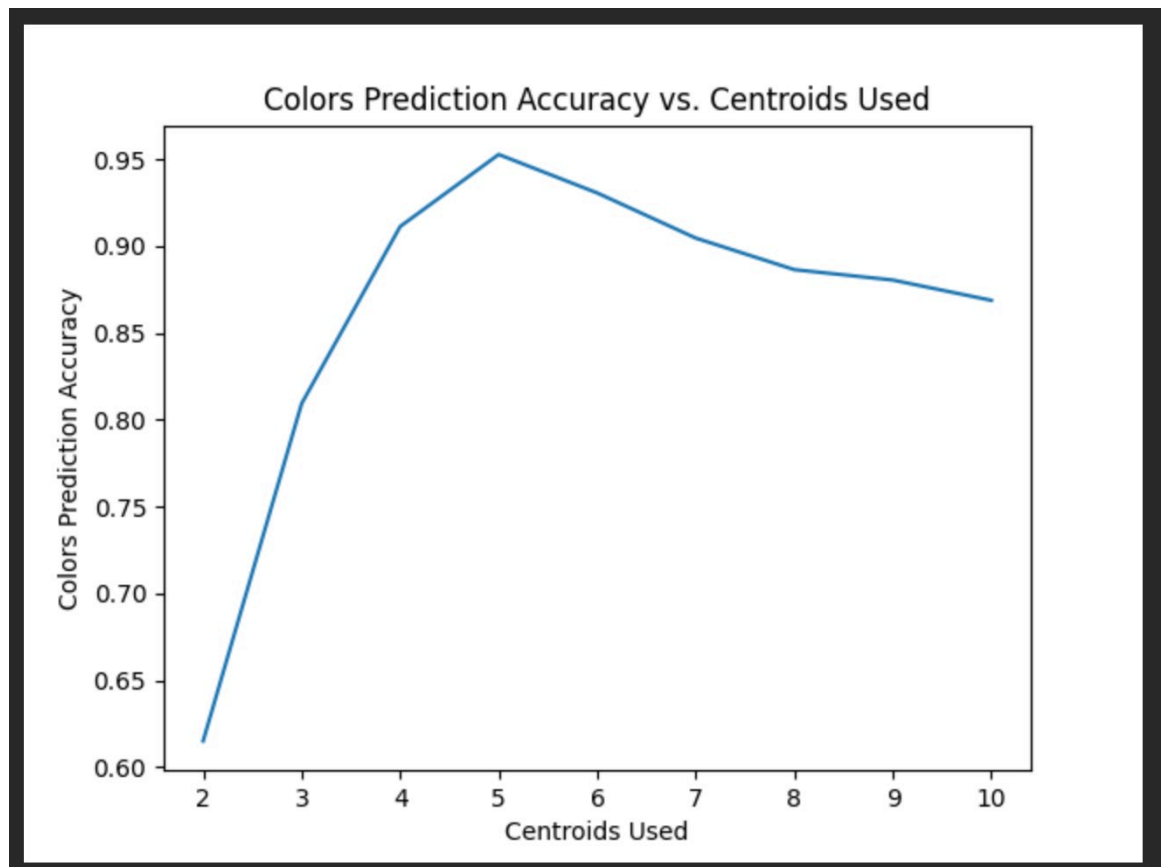




The variety of colours in the original image dictates how many centroids we need to have in order to replicate the image properly. Contrary to the image above, which has plenty of primary colours, here is another image for which a smaller number of centroids would be enough to represent it:



Here is a graphic that shows how the number of centroids used affect the model's accuracy:



Choosing the right number of centroids is essential in obtaining good performance and also fast computation. However, since every image can have varying colour distributions, we are using an algorithm that finds the best number of centroids for each image.

Result obtained:

Train set	99%
Test set	39%

! Not to be misunderstood !

KMeans is an unsupervised algorithm, so there is no such thing as a Training and Test set. The role of this table is to portray the fact that there is such a high discrepancy between the accuracy on the two data. That is because the GT for the training set includes all the colours that appear in the image (even model's skin and background), while the test set's GT only includes the colour of the clothing item. Since our KMeans algorithm also identifies the colours in the picture that correspond to background and skin, this explains the high accuracy on the Train set and low accuracy in the Test set.

4.Improvements

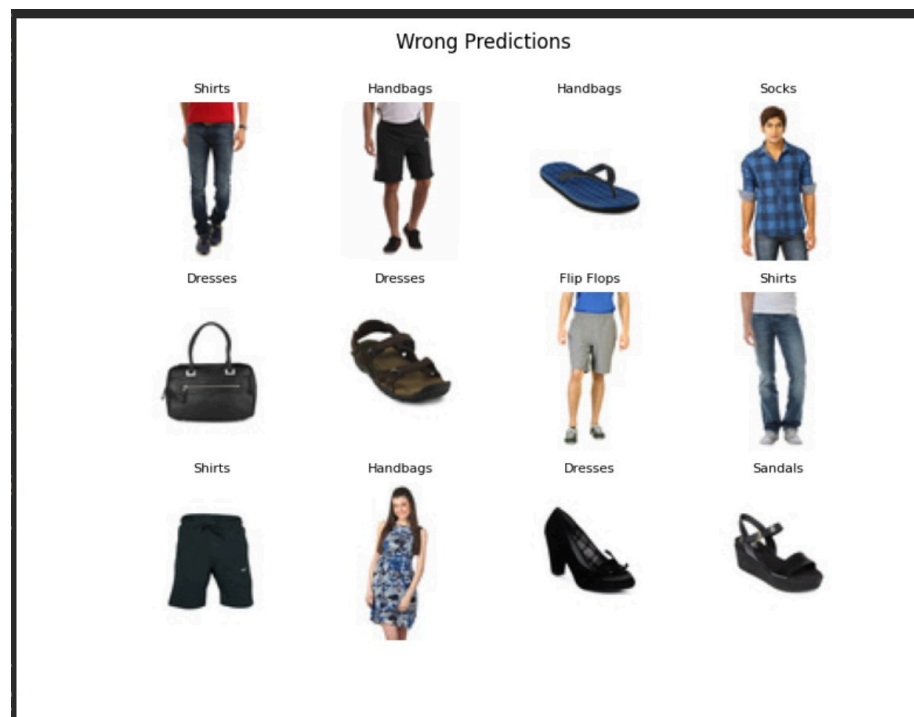
4.1.Enhancing the shape classification performance

- Mean Value + Difference
- a) Modifications: Mean Pixel Value as the Feature and Absolute Value of the Difference as the Distance Function

```
# Mean
self.train_data = np.array([np.mean(train) for train in self.train_data])
test_data = np.array([np.mean(test) for test in test_data])
test_data = np.abs(test_data[:, np.newaxis] - self.train_data)
sorted_matrix = test_data.argsort(axis=1)[: , :k]
self.neighbors = np.array([self.labels[row] for row in sorted_matrix])
```

- b) Results:

By running `get_shape_accuracy` with the above modifications, we get a 40.5% accuracy for one neighbour and 27% for 5, which is terrible compared to the 91% accuracy of the baseline. Here are some misclassifications obtained with `visualize_retrieval`:



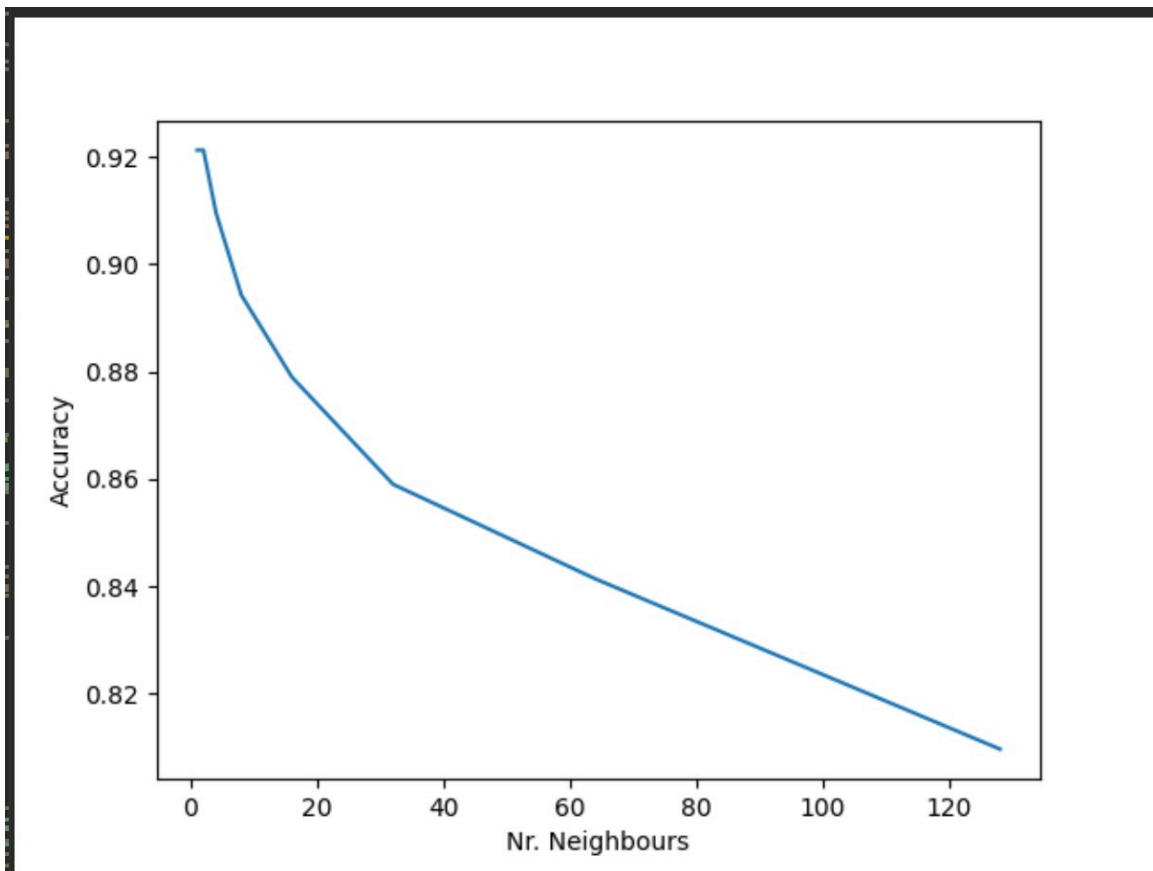
The misclassifications are not even that obvious anymore. This modification is a failure. The reason for the bad classification is that we removed the model's ability of identifying shapes' borders and now it treats an image as a single numerical value.

- *Cosine Distance*

a) Modification: Keeping Pixels' Intensity as the Features and changing the Distance Function to Cosine Distance

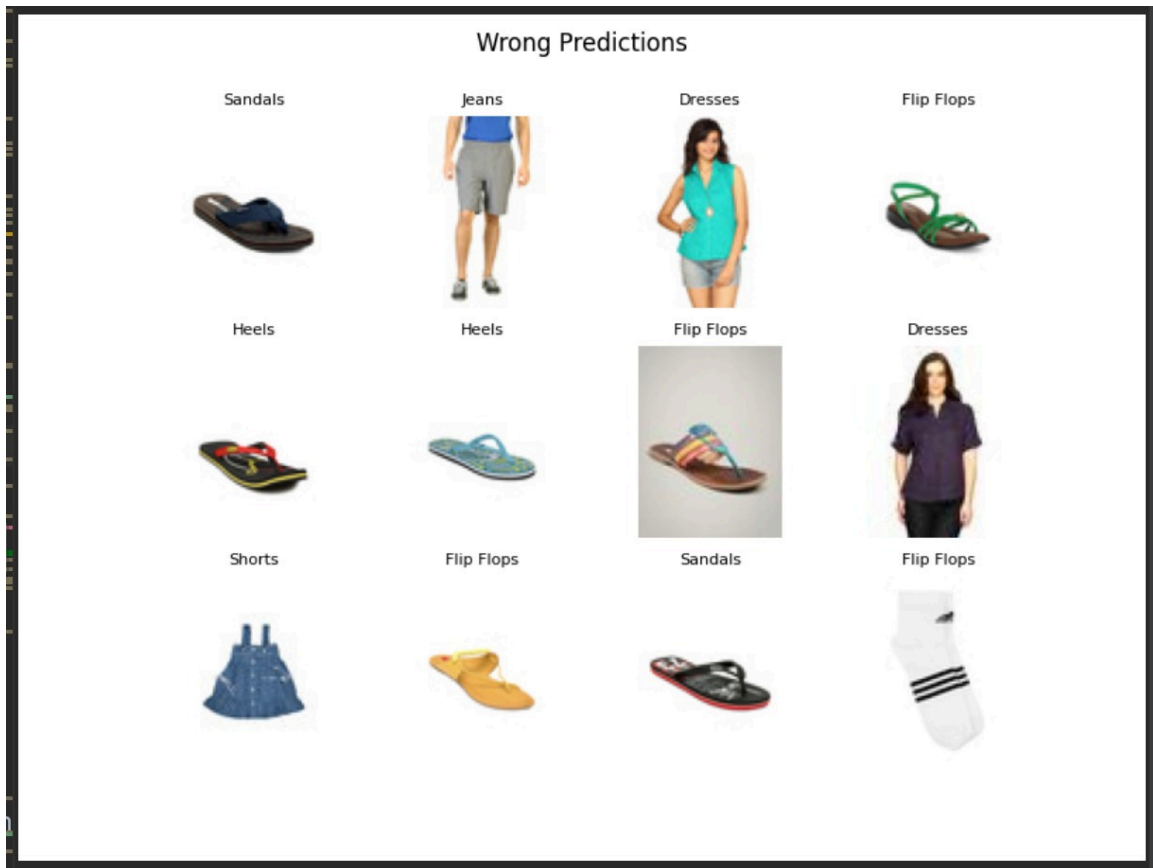
b) Results:

By running the algorithm with the above modifications, `get_shape_accuracy` renders 92.3% accuracy for 1 neighbour and 89.7% for 10 neighbours. Below is the plot obtained with the qualitative function `create_plot` with the parameters provided by the quantitative function `knn_neighbours_vs_accuracy`:



c) Conclusion:

Compared with the plot of the Baseline Method which uses Euclidean Distance, we can conclude that both of them follow a similar downward trend on this metric. Although the accuracy improves by 0.5%, the same problem persists with this enhancement as well: inability to distinguish between flip-flops, heels and sandals.



4.2. Pre-processing the images

Hypothesis:

The GT for the training set includes all the colours that appear in the image (even model's skin and background), while the test set's GT only includes the colour of the clothing item. Since our KMeans algorithm also identifies the colours in the picture that correspond to background and skin, this explains the high accuracy on the Train set and low accuracy in the Test set.

Solution Design:

In order to obtain a better accuracy on predicting **only** the colour of the clothing item, we will begin with pre-processing the image before running KMeans.

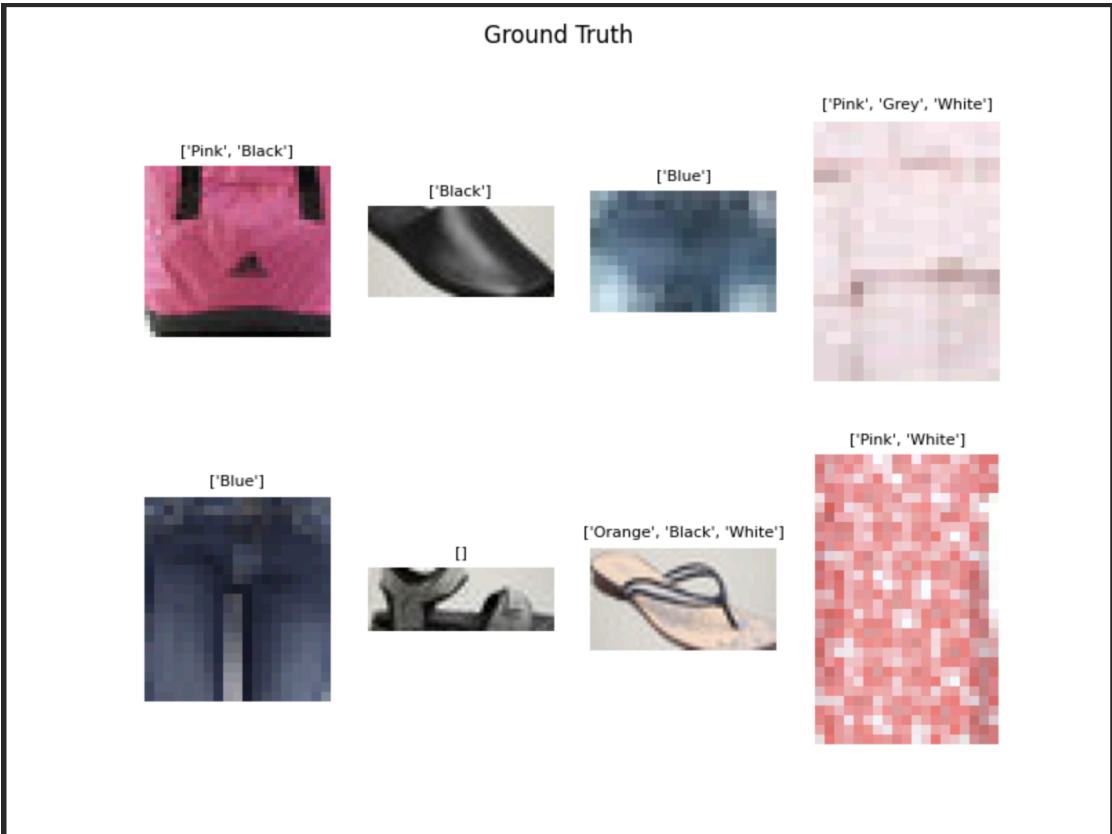
We will implement this by using the `read_extended_dataset` function provided. This will shift the focus on a smaller window containing the important part of the image.

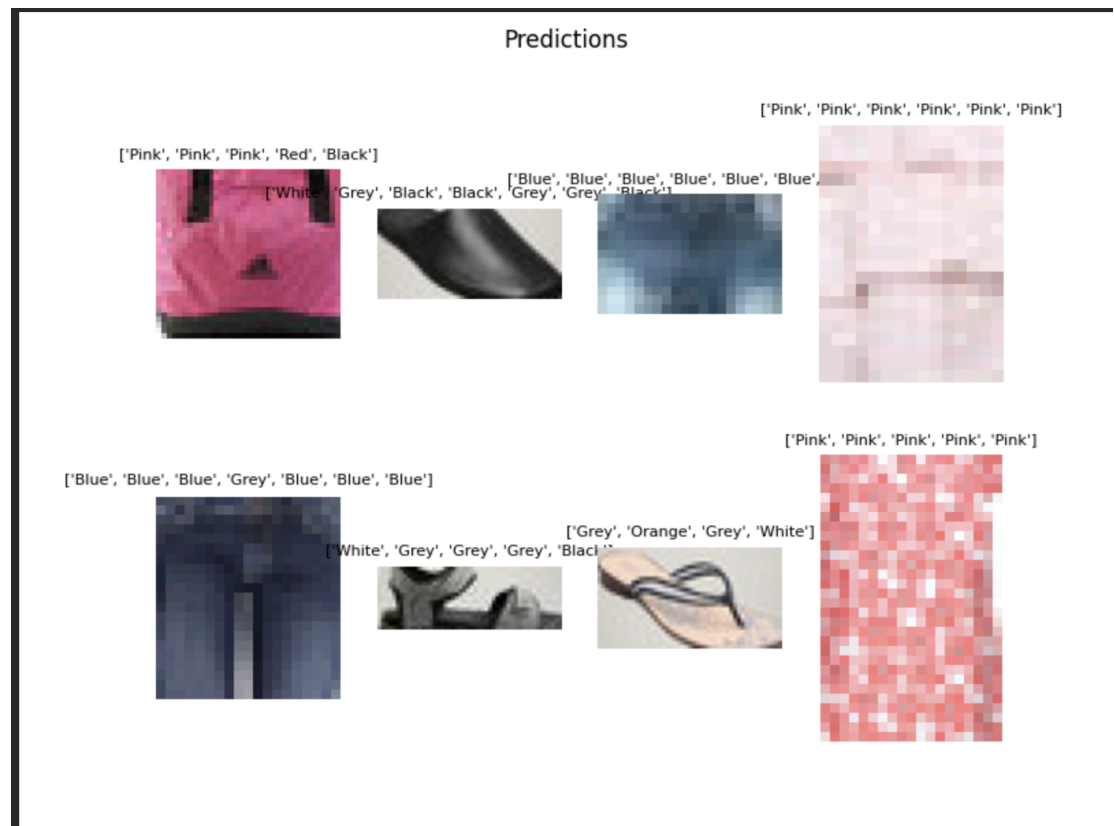
Result obtained by the Baseline:

Train set	99%
Test set	39%

- a) Modification: Using a smaller data_set that is made out of cropped photos
- b) Results:

The result provided by get_color_accuracy is 57%, which means an 18% increase.



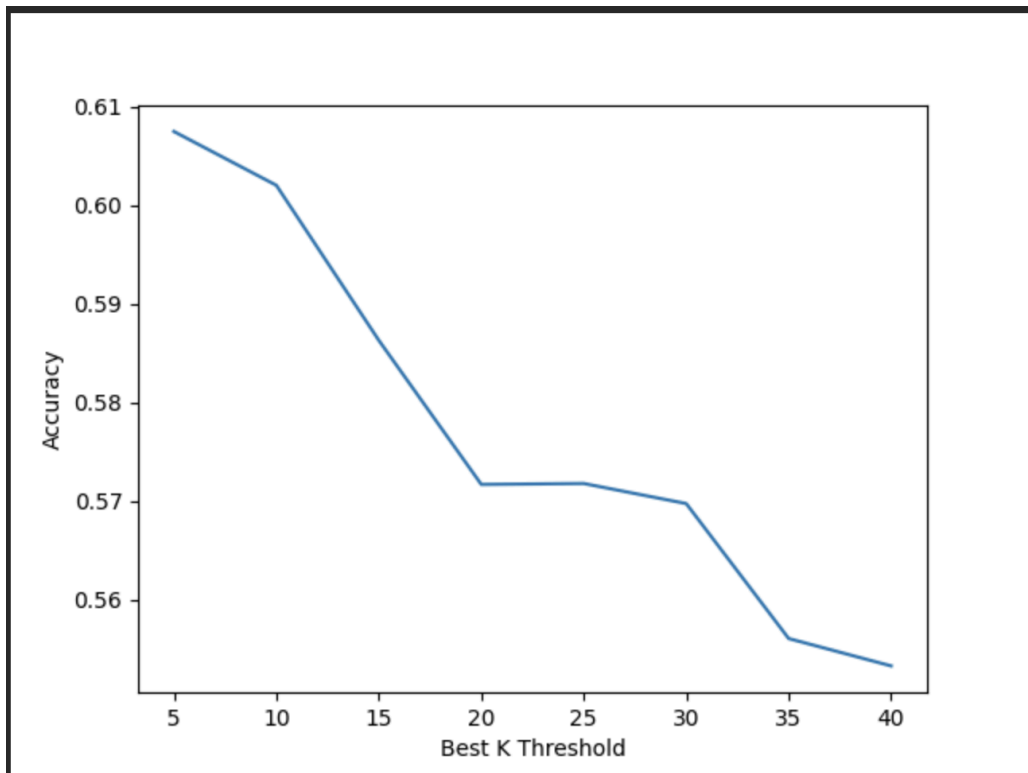


c) Conclusion:

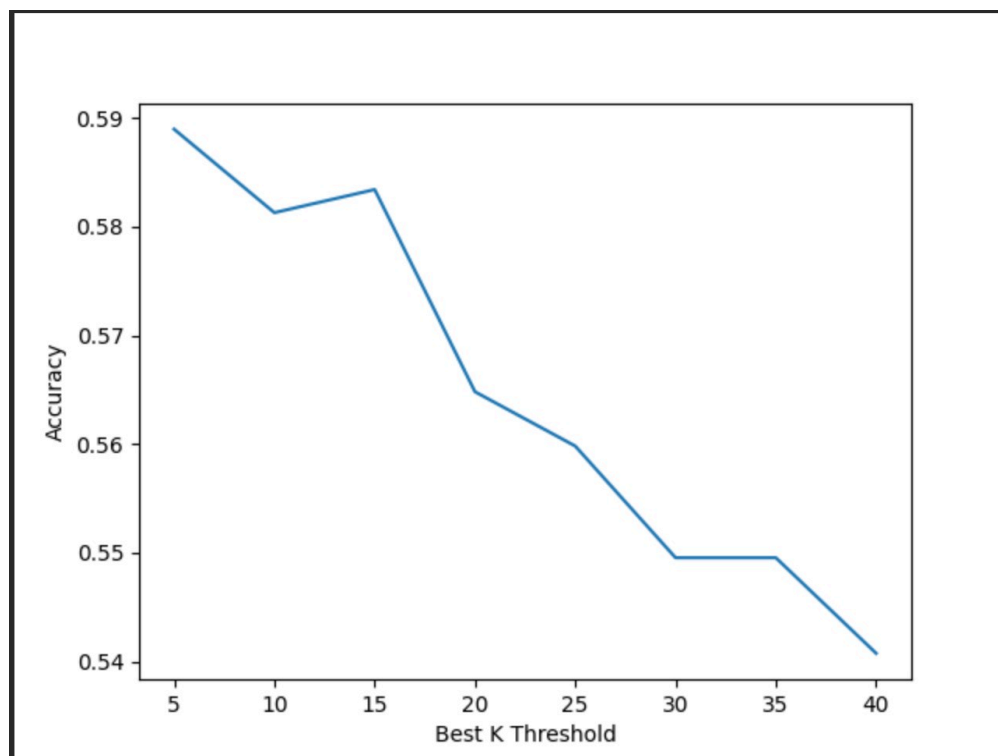
Shifting the algorithm's focus on the important part of the image seems to be working fine. However, as can be seen in the tagging above, our predictions contain too many tags compared to the GT. This is because of the high number of centroids, the Distance function used (WCD) and the KMeans find_best_k threshold value. In the next section we will try to improve our results by testing variations of the features above.

4.3. Testing different parameters for KMeans find_best_k

Contrary to our expectations, the best_k_threshold did not play such a big role in the accuracy of the prediction:



The next parameter we are going to test is the distance function for selecting the number of centroids in the best_k algorithm. Currently, up until now we were using the WCD and now we are implementing the Fischer discriminator which uses both Intra-class distance and Inter-class distance. Here are the results of the modification:



Neither this modification enhances the performance.

5.Global Conclusions

The results obtained by improving the algorithms are 92% for KNN (+1% than Baseline) and 60% on KMeans (+21% than Baseline).

Our conclusions on the KNN shape classification algorithm is that it is close to its maximum potential accuracy, due to the impossibility to distinguish between flip-flops, heels and sandals, which are very similar to each other. One possible boost would be providing more training samples of these categories.

Regarding KMeans, although there is a great improvement in accuracy, we still believe that the performance of the model is far away from its potential. However, it is worth noting that the colour labels on this data_set were assigned by the human eye, which means they are prone to imprecision and there is room for bias.

Colour labels of the part of the piece that is given to the subimage. These labels have been assigned by a human, not by an algorithm. (extracted from the project description)

In order to prove the hypothesis, we attach below the tags obtained for the following image.

Ground Truth Tag

['Orange', 'Black', 'Grey', 'White']



Our Tag (Colour + Percentage)

White 0.013888888888888888
White 0.24691358024691357
Orange 0.016975308641975308
White 0.06558641975308642
White 0.023919753086419752
Orange 0.035493827160493825
Orange 0.06481481481481481
Orange 0.36651234567901236
Red 0.038580246913580245

Red 0.022376543209876542
Orange 0.019290123456790122
Pink 0.020061728395061727
White 0.030864197530864196
Red 0.024691358024691357
White 0.010030864197530864

For this specific image, we tried our best to make the model see the colours of Black and Grey by increasing the number of centroids to 15 (we basically made it capture more specific colours). However, the model is still unable to find those colours in the image. This may be due to the fact that the image has a low quality (80x60) and some colours are blended together or due to the different method of estimating colours (our model is using the function `get_color` for estimating the colour of each centroid, while the human eye may estimate the principal colours to different colours classes).

Cases such as the one presented above are widely spread in the data set and since we did not figure out a solution to this problem we consider this fact a limitation of the model.